Process State Diagram, Schedulers, and Context Switch

Lecture Notes — Lecture 6

How the OS Tracks and Switches Processes

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

October 29, 2026

Concepts That Click

6.1 One CPU, Three Programs

A lab machine has *one* CPU core. On it, `gcc` is compiling a large project, a text editor is waiting for the next keystroke, and a backup job has been queued for the night. To the person at the keyboard, all three feel as if they are running. The question that organizes this chapter is what that feeling hides: *at any instant, how many of them actually are running — and how does the operating system decide which one runs, remember where the others stopped, and switch between them?*

We give the three programs names and keep them for the whole chapter: `P1` is `gcc`, which has long CPU bursts; `P2` is the editor, which has short bursts and spends most of its time waiting; `P3` is the backup job, which has been submitted but not yet admitted. On one core, at most **one** process is running at any instant. Everything else in this chapter explains how the kernel manages the others.

What we already know. Lecture 5 defined a **process** as a program in execution: its code, data, stack, and heap, together with the CPU state that goes with them. The kernel records each process in a **process control block** (PCB), which holds the process state, the program counter (`PC`), the registers, scheduling information, memory-management information, accounting information, and the list of open files. The PCB is what lets a process *stop* and later *resume* exactly where it left off (Silberschatz et al., Ch. 3 §3.1, pp. 101–103 [1]). The PCB has a *state* field. This chapter asks which values that field can hold, which events change it, and who changes it.

Roadmap. Four questions frame the chapter:

1. **Which states can a process be in?** The five-state diagram and its legal transitions (Section 6.2).
2. **Where does the kernel keep processes?** The job, ready, and device queues, and the queueing diagram (Section 6.3).
3. **Who moves processes between queues?** The long-term, short-term, and medium-term schedulers (Section 6.4).
4. **What does a switch cost?** The context switch, the dispatcher, and its overhead (Section 6.5).

The division of labour is worth stating once: the states are the vocabulary, the queues are the data structures, the schedulers are the policies, and the context switch is the mechanism.

6.2 The Process State Diagram

Reading: Silberschatz et al., Ch. 3 §3.1, p. 103; Stallings, Ch. 3 §3.2, p. 111.

A process does not simply "run." It is created, waits its turn, runs, blocks, and finishes, and the kernel tracks every one of those steps in a single field of the PCB.

Definition (Process state). *The state of a process is the PCB field that records its current activity. A process is in exactly one of five states at any instant.*

State	Meaning
`new`	being created; the PCB is being set up
`ready`	in memory, able to run, waiting only for a CPU
`running`	its instructions are being executed on a CPU
`waiting`	blocked until some event occurs (I/O completion, a signal)
`terminated`	finished; its resources are being reclaimed

Table 6.1: The five process states (Silberschatz et al., Ch. 3 §3.1, p. 103).

The names vary between books. Stallings calls `waiting` "blocked" and `terminated` "exit" (Ch. 3 §3.2, p. 111 [2]); the states themselves are the same.

Figure 6.1 draws the states as nodes and the permitted changes as arrows. Read it from left to right: a process is admitted from `new` into `ready`; the scheduler dispatches it to `running`; from `running` it can be interrupted back to `ready`, block in `waiting`, or exit to `terminated`; and a waiting process returns to `ready` when its event completes.

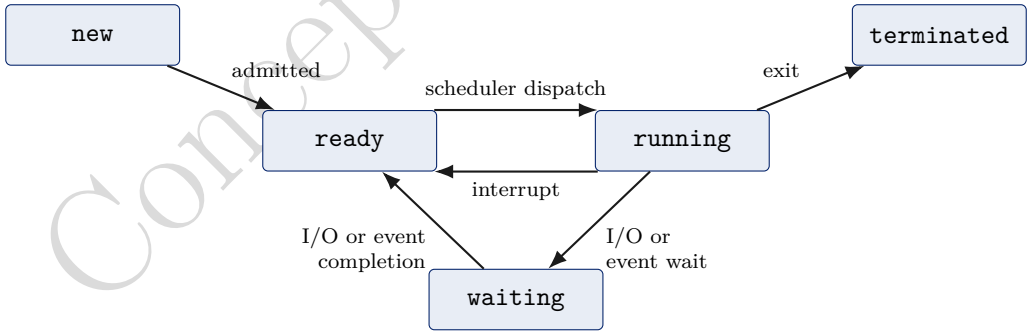


Figure 6.1: The five-state process diagram — after Silberschatz et al., Ch. 3 §3.1, p. 103 [1].

6.2.1 The six legal transitions

Each arrow in Figure 6.1 has a triggering event and an agent that carries it out. Table 6.2 lists all six.

Two transitions are conspicuously absent, and both absences are deliberate. There is no `waiting` → `running` arrow: a process whose event has just occurred must queue for the CPU like every

Concepts That Click

Transition	Triggering event	Who acts
`new` → `ready`	admitted to the system	long-term scheduler
`ready` → `running`	chosen for the CPU	short-term scheduler + dispatcher
`running` → `ready`	timer interrupt / preemption	kernel (interrupt handler)
`running` → `waiting`	I/O request, `wait`, sleep	the process itself (system call)
`waiting` → `ready`	the awaited event occurs	kernel (I/O-completion interrupt)
`running` → `terminated`	`exit` or fatal error	the process / the kernel

Table 6.2: The six legal transitions, their triggers, and their agents.

other ready process. There is no `ready` → `waiting` arrow: only a running process can issue the request it would wait on, so a process that is not running has nothing to block on.

The separation in the first case matters. Suppose `P2` is waiting for the disk, the disk interrupt arrives, and the CPU happens to be idle. Does `P2` go straight to `running`? No: in the diagram it goes `waiting` → `ready`, because the interrupt handler only marks it runnable. The short-term scheduler then chooses among *all* ready processes; with an idle CPU it may pick `P2` almost immediately, but it is still a separate decision. Keeping "wake up" and "get the CPU" as two steps is exactly what lets the scheduler enforce a policy.

Example 6.1 (The editor, for one keystroke)**.** *Follow `P2`, the editor, through a single keystroke.*

1. *`P2` is `running`. It calls `read` on the keyboard, but no key is pending. Transition: `running` → `waiting`.*
2. *The kernel places `P2`'s PCB on the keyboard's device queue and runs another process. `P2` now consumes no CPU time.*
3. *You press a key. The keyboard interrupt handler finds `P2` waiting for it. Transition: `waiting` → `ready`.*
4. *`P2`'s PCB joins the ready queue. It does* not *run yet; whatever was running continues.*
5. *The short-term scheduler picks `P2`, and the dispatcher loads its context. Transition: `ready` → `running`.*
6. *`P2` echoes the character and calls `read` again. Transition: `running` → `waiting`.*

Result: *each keystroke costs the editor three transitions (`waiting` → `ready` → `running` → `waiting`), and between keystrokes the editor spends almost all of its life in `waiting`.*

6.2.2 How many processes can be in each state?

With N CPU cores, at most N processes can be `running`. Any number may be `ready` or `waiting` at once. On our one-core lab machine, a snapshot at one instant might show `P1` `running`, `P2` `waiting` on the keyboard, and `P3` `new`, still in the batch queue.

Example 6.2 (A snapshot of 40 processes on 4 cores)**.** *A 4-core machine has 40 processes. At most 4 are `running`. If 9 are `ready`, then the remaining $40 - 4 - 9 = 27$ are `waiting` (or `new` or `terminated`). This is typical of desktops: most processes are waiting most of the time.*

Concepts That Click

6.2.3 Suspended processes and the seven-state model

Memory is finite. If every process in memory is `waiting`, the CPU idles even though more work is queued on disk. The remedy is to **swap out** a process: copy its image to disk and free its memory. Such a process is **suspended**. Stallings (Ch. 3 §3.2, p. 111) splits each of `ready` and `waiting` into an in-memory version and a suspended version, giving a **seven-state model** with two new states:

- `ready/suspend`: the process is on disk but able to run once it is brought back into memory;
- `blocked/suspend`: the process is on disk *and* still waiting for an event.

Figure 6.2 shows the model. The top row lives in memory; the bottom row has been swapped out to disk. A blocked process can be suspended; if its event occurs while it is on disk it moves to `ready/suspend` without being brought back; and either suspended state can be activated back into memory.

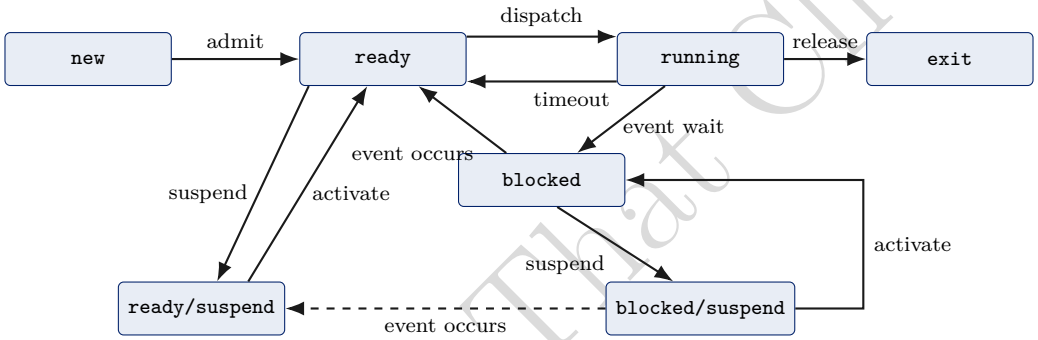


Figure 6.2: The seven-state model with suspension — after Stallings, Ch. 3 §3.2, p. 111 [2]. Top row: in memory. Bottom row: swapped out to disk.

6.3 Scheduling Queues

Reading: Silberschatz et al., Ch. 3 §3.2, pp. 105–110.

A state is just a label. To act on it, the kernel keeps processes of each state in queues, and the queues are where scheduling happens. Silberschatz et al. describe three kinds:

Definition (Scheduling queues). *The **job queue** holds every process in the system (in the batch-oriented description, jobs submitted and awaiting admission). The **ready queue** holds the processes in main memory that are `ready`, usually as a linked list of PCBs. Each device has a **device queue** holding the processes `waiting` for that device.*

The implementation is simple. A queue header holds pointers to the first and last PCB, and each PCB has a pointer field to the next PCB in the same queue. Moving a process between queues moves a pointer, not the process. Figure 6.3 shows a ready queue holding `P1`, `P4`, and `P5` and a disk queue holding `P2` and `P6`. A PCB sits on exactly one of these queues at a time.

6.3.1 The queueing diagram

Figure 6.4 is the standard picture of how a process moves among these queues. A new process is put in the ready queue and waits there until it is dispatched. Once on the CPU, it leaves for exactly one of four reasons:

Concepts That Click

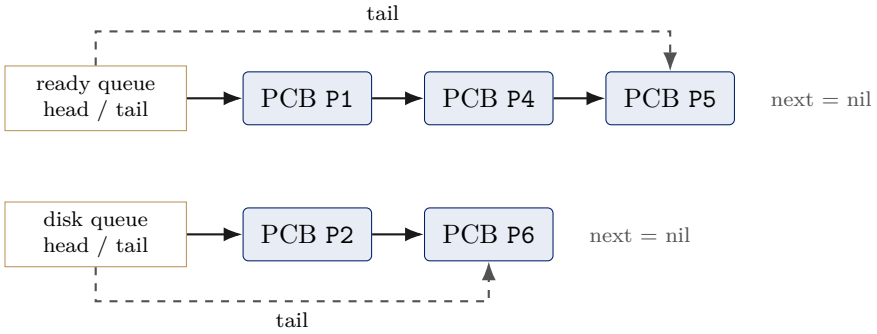


Figure 6.3: The ready queue and a device queue as linked lists of PCBs — after Silberschatz et al., Ch. 3 §3.2, pp. 105–110.

1. it issues an **I/O request** and is placed on a device queue;
2. its **time slice expires**, and a timer interrupt returns it to the ready queue;
3. it **creates a child** process and may wait for the child to terminate;
4. it is **removed by an interrupt**, or waits for an event.

In every case except termination, the path leads back to the ready queue. In our running example, `P2` (the editor) leaves by route 1 almost every burst, `P1` (`gcc`) leaves mostly by route 2, and a shell running `gcc` leaves by route 3.

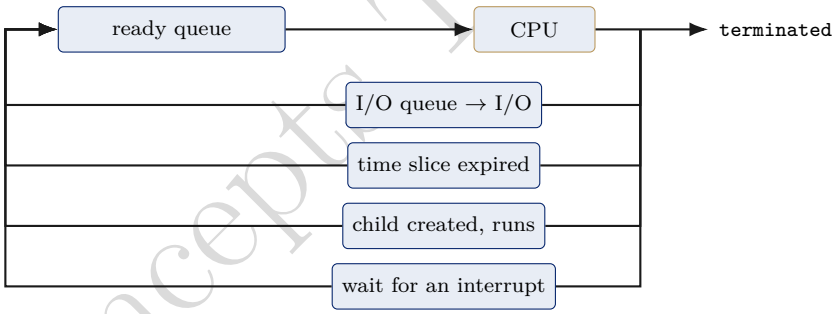


Figure 6.4: The queueing diagram — after Silberschatz et al., Ch. 3 §3.2, pp. 105–110. Every path out of the CPU, except termination, leads back to the ready queue.

Example 6.3 (Queue membership over time)**.** *Table 6.3 follows the three processes through six events on the one-core machine.*

Step by step: at t_1 the timer preempts `P1`, which joins the ready queue, and `P2` is dispatched. At t_2, `P2` blocks on `read`, so it moves to the keyboard queue and `P1` is dispatched again. At t_3 the keyboard interrupt moves `P2` to the ready queue, where nothing is ahead of it — but `P1` keeps the CPU until its slice expires at t_4. At t_5 the backup job `P3` is admitted and joins the tail of the ready queue behind `P1`. Check: *every row has exactly one running process, and each PCB appears in exactly one place.*

6.4 Schedulers

Reading: Silberschatz et al., Ch. 3 §3.2, pp. 105–110.

Concepts That Click

Time	Event	Running	Ready queue	Keyboard queue
t_0	start	`P1`	`P2`	—
t_1	`P1`'s time slice expires	`P2`	`P1`	—
t_2	`P2` calls `read`	`P1`	—	`P2`
t_3	key pressed (interrupt)	`P1`	`P2`	—
t_4	`P1`'s time slice expires	`P2`	`P1`	—
t_5	`P3` admitted	`P2`	`P1`, `P3`	—

Table 6.3: Queue membership for `P1`–`P3`.

Queues hold the processes; schedulers decide who moves between them. An operating system has up to three schedulers, and they work on very different timescales, as Table 6.4 summarizes.

Scheduler	Moves a process	Runs	Controls
Long-term (job)	job pool → ready queue (`new` → `ready`)	seconds to minutes	degree of multiprogramming
Short-term (CPU)	ready queue → CPU (`ready` → `running`)	every few ms	who runs next
Medium-term	memory ↔ disk (swap out / swap in)	occasionally	memory pressure, mix

Table 6.4: The three schedulers.

Concepts That Click

6.4.1 The long-term scheduler

Definition (Long-term scheduler). *The long-term (job) scheduler selects processes from the job pool on disk and loads them into memory for execution. It controls the degree of multiprogramming n, the number of processes in memory.*

In a stable system the average rate of process admission must equal the average rate of departure, so the long-term scheduler may need to be invoked only when a process leaves. Because it runs rarely, it can afford a careful choice. Batch systems have one; time-sharing systems such as UNIX and Windows often have *none*, and every new process goes straight to the short-term scheduler.

The most important choice the long-term scheduler makes is the **process mix**. An **I/O-bound** process spends more time doing I/O than computing and has many short CPU bursts — `P2`, the editor, is one. A **CPU-bound** process spends most of its time computing and has few, long bursts — `P1`, `gcc`, is one. If every admitted process is I/O-bound, the ready queue is almost always empty and the CPU idles; if every process is CPU-bound, the device queues are empty and the devices idle. A good mix keeps both busy (Silberschatz et al., Ch. 3 §3.2, pp. 105–110).

The absence of a long-term scheduler has a visible consequence. When students open dozens of browser tabs until a Linux laptop slows to a crawl, no scheduler refuses them: admission control would be the long-term scheduler's job, but a time-sharing system typically has none. Every new process goes straight to the ready queue, and stability depends on users noticing and quitting programs. Memory pressure is handled instead by swapping or paging — a medium-term mechanism — which is why the machine slows down rather than refusing work.

6.4.2 The short-term scheduler

Definition (Short-term scheduler). *The short-term (CPU) scheduler selects one process from the ready queue and allocates the CPU to it.*

It runs very often: at least once every time slice (typically every few tens of milliseconds) and after every block. It must therefore be fast, because its own running time is pure overhead.

Example 6.4 (Scheduler overhead)**.** *The short-term scheduler takes* 1 *ms to decide which process to run, and the chosen process then runs for* 10 *ms. Each cycle is* $10 + 1 = 11$ *ms long, of which* 1 *ms is spent scheduling. The fraction of CPU time spent scheduling is therefore*

$$\frac{1}{10+1} = \frac{1}{11} \approx 9\%.$$

How the short-term scheduler chooses — FCFS, SJF, RR, and the rest — is the subject of Lectures 8 and 9.

6.4.3 The medium-term scheduler

Definition (Medium-term scheduler). *The medium-term scheduler removes a process from memory (swaps it out) to reduce the degree of multiprogramming, and later reintroduces it (swaps it in) so that it continues where it left off.*

Swapping is used to improve the process mix or to free memory when demand exceeds supply. Figure 6.5 adds it to the queueing picture.

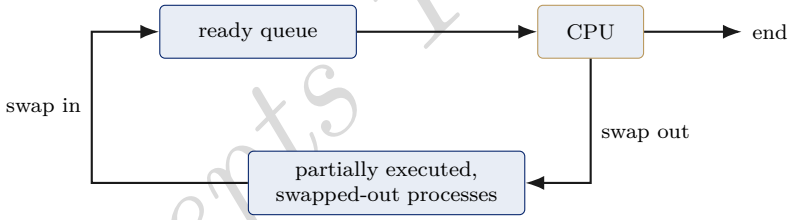


Figure 6.5: The medium-term scheduler adds a swap-out/swap-in loop — after Silberschatz et al., Ch. 3 §3.2, pp. 105–110.

6.4.4 Schedulers on the state diagram

Putting the two diagrams together shows exactly which transitions each scheduler drives. The long-term scheduler drives `new` → `ready` (admission). The short-term scheduler drives `ready` → `running` (dispatch) and, with preemption, also decides `running` → `ready`. The medium-term scheduler drives `ready` ↔ `ready/suspend` and `blocked` ↔ `blocked/suspend`. Two transitions are *not* a scheduler's decision: `running` → `waiting` is caused by the process's own request, and `waiting` → `ready` is caused by the completion interrupt.

6.5 The Context Switch

Reading: Silberschatz et al., Ch. 3 §3.2, p. 110; Stallings, Ch. 3 §3.4, p. 135; Bovet & Cesati, Ch. 3, pp. 79–130.

Concepts That Click

The short-term scheduler picks the next process. Handing it the CPU means saving one process's context and restoring another's.

Definition (Context and context switch). *The context of a process is the CPU and kernel state needed to resume it: register values, `PC`, process state, and memory-management information, all represented in its PCB. A context switch saves the context of the current process into its PCB and restores the saved context of the process scheduled to run next.*

A context switch has two halves, a **state save** and a **state restore**. It is triggered by an interrupt, a trap, or a system call that blocks. Figure 6.6 shows two processes alternating: an interrupt moves the CPU from `P0` to the kernel, which saves PCB_0 and reloads PCB_1; later a system call from `P1` triggers the reverse. While the kernel switches, neither process executes.

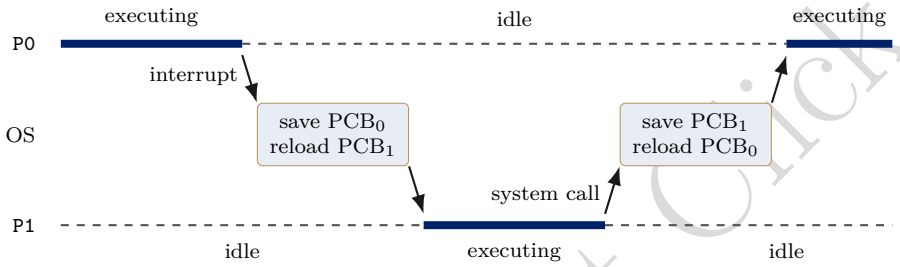


Figure 6.6: A context switch on a timeline — after Silberschatz et al., Ch. 3, pp. 103–110. The gold boxes are pure overhead.

6.5.1 The steps of a context switch

Stallings (Ch. 3 §3.4, p. 135) lists the full sequence of process-switch steps; Bovet & Cesati (Ch. 3, pp. 79–130 [3]) describe Linux's implementation. In outline, switching from `P0` to `P1`:

1. An interrupt or trap enters the kernel; the hardware saves the user `PC` and flags.
2. The kernel saves the remaining registers of `P0` into PCB_0.
3. It updates PCB_0's state (`ready` or `waiting`) and links PCB_0 onto the right queue.
4. The short-term scheduler selects `P1`.
5. The kernel sets PCB_1's state to `running` and switches memory-management state (page-table base, TLB as needed).
6. It restores `P1`'s registers from PCB_1 and returns to user mode at `P1`'s saved `PC`.

6.5.2 The dispatcher

Definition (Dispatcher). *The dispatcher is the kernel module that gives control of the CPU to the process selected by the short-term scheduler. The time it takes to stop one process and start another is the dispatch latency.*

The dispatcher switches context, switches to user mode, and jumps to the proper location in the user program. The scheduler is the *policy* (which process?); the dispatcher is the *mechanism* (make it run). Every dispatch involves a context switch. The dispatcher is previewed here and developed in Lecture 8 (Silberschatz et al., Ch. 5 §5.1, from p. 183).

Concepts That Click

6.5.3 A mode switch is not a context switch

A **mode switch** goes user → kernel → user for the *same* process — for example, a `getpid` system call. A few registers are saved and restored, and the PCB state does not change. A **context (process) switch** moves the CPU to a *different* process: a full register save, a PCB state update, a queue move, and a memory-map switch. Every context switch includes mode switches, but most mode switches cause no context switch; an interrupt can be handled and return to the interrupted process with no process switch at all (Stallings, Ch. 3 §3.4, p. 135).

The test is simple: *does the caller keep the CPU when the kernel returns?* A process that calls `getpid` does — the kernel answers and returns, a mode switch only. A process that calls `read` on an empty pipe does not: it must block (`running` → `waiting`), so the kernel switches to another process.

6.5.4 The cost of switching

A context switch is pure overhead: the system does no useful work while switching. Its speed depends on the hardware — memory speed, the number of registers to copy, and special instructions such as a single instruction to load or store all registers. A typical cost is a few microseconds, plus an indirect cost from cold caches and TLB. Some processors provide multiple register sets, so a switch simply changes a pointer to the current set. The general rule (Silberschatz et al., Ch. 3 §3.2, p. 110): the more complex the operating system and its memory management, the more context must be saved and the more each switch costs.

If a switch costs s and each process runs for a time slice q before being switched out, each cycle lasts $q + s$, of which s is overhead:

$$\text{overhead fraction} = \frac{s}{q+s}.$$

Example 6.5 (Overhead of switching)**.** *Let $s = 5$ μs. The number of switches per second is approximately $1/(q+s)$.*

- $q = 10\,ms = 10000$ μs*:* $1/(10005\ \mu s) \approx 99.95$ *switches/s; overhead* $5/10005 \approx 0.05\%$*.*
- $q = 1\,ms = 1000$ μs*:* ≈ 995 *switches/s; overhead* $5/1005 \approx 0.50\%$*.*
- $q = 100$ μs*:* ≈ 9524 *switches/s; overhead* $5/105 \approx 4.76\%$*.*
- $q = 10$ μs*:* ≈ 66667 *switches/s; overhead* $5/15 \approx 33.3\%$*.*

Conclusion: *shrink the slice and the CPU spends more of its life switching than working.*

Example 6.6 (`P1` and `P2` on one core)**.**

1. *$t = 0$: `P1` (`gcc`) is running; `P2` is waiting on the keyboard.*
2. *$t = 3\,ms$: a key is pressed. The interrupt moves `P2` to `ready`. This is a mode switch only: `P1` resumes.*
3. *$t = 10\,ms$: `P1`'s slice expires. Context switch #1: save PCB_1, `P1` → `ready`; restore PCB_2, `P2` → `running`.*
4. *$t = 10.2\,ms$: `P2` echoes the key and calls `read`. Context switch #2: `P2` → `waiting`; `P1` → `running`.*

Count: *two context switches and three kernel entries (interrupt, timer, system call) in* $10.2\,ms$*. With $s = 5$ μs the switching cost is $2 \times 5 = 10$ μs, and $10/10200 \approx 0.1\%$ of the interval.*

Concepts That Click

6.6 Putting It Together

States, queues, schedulers, and the context switch form one machine. Following the backup job `P3` through its whole life uses every part of it.

Example 6.7 (`P3`'s life, end to end). *1. Submitted to the batch queue: `new`, in the job queue.*

2. *The long-term scheduler admits it at night: `new` → `ready`, PCB on the ready queue.*
3. *The short-term scheduler picks it and the dispatcher context-switches to it: `ready` → `running`.*
4. *It writes to the backup disk: `running` → `waiting`, on the disk queue; the kernel context-switches away.*
5. *Memory is short, so the medium-term scheduler swaps it out: `blocked` → `blocked/suspend`.*
6. *The write completes: `blocked/suspend` → `ready/suspend`; later it is swapped in: → `ready`.*
7. *It is dispatched again, finishes, and calls `exit`: `running` → `terminated`.*

All three schedulers and both diagrams appear in one life.

6.6.1 Common misconceptions

- "A woken process starts running immediately." It goes `waiting` → `ready` and queues.
- "Every system call is a context switch." Most are mode switches; only blocking or preemption switches process.
- "Every OS has a long-term scheduler." Time-sharing systems often have none.
- "The ready queue holds waiting processes." Waiting processes are on device queues.

6.6.2 Summary

- A process is in one of five states — `new`, `ready`, `running`, `waiting`, `terminated` — and six transitions are legal. Suspension adds two more states.
- The kernel keeps PCBs on a job queue, a ready queue, and device queues; the queueing diagram shows every path.
- The long-term scheduler sets the degree of multiprogramming; the short-term scheduler picks the next process every few milliseconds; the medium-term scheduler swaps.
- A context switch saves one PCB and restores another; it is pure overhead, a fraction $s/(q+s)$ of the CPU.
- A mode switch is not a context switch.

Concepts That Click

Next. This chapter dealt with processes that already exist. The next lecture asks how processes come to exist and how they end — `fork`, `exec`, `wait`, `exit`, and the process tree. Reading: Silberschatz et al., Ch. 3 §3.3 [1]; Stallings, Ch. 3 §3.4 [2]; Bovet & Cesati, Ch. 3 [3].

For this chapter, the page-cited sources are Silberschatz et al., 8th ed., Ch. 3 §3.1 (process states, p. 103) and §3.2 (process scheduling: queues, schedulers, context switch, pp. 105–110); Stallings, 6th ed., Ch. 3 §3.2 (two-, five-, and seven-state models, p. 111) and §3.4 (process control, mode and process switching, p. 135); and Bovet & Cesati, 3rd ed., Ch. 3 (processes and the process switch, pp. 79–130). All are page-verified through each book's `index.md` in `master_supporting_docs/CS502/supporting_books/`.

Self-Check

Exercises

1. For each pair, state whether the transition is legal in the five-state model and, if it is, name the event and the agent: (a) `ready` → `running`; (b) `waiting` → `running`; (c) `running` → `waiting`; (d) `ready` → `waiting`.

2. An 8-core server has 120 processes. At one instant 8 are running, 17 are ready, and 2 are new. How many are waiting, assuming none is terminated? What is the largest possible number of running processes?

3. A short-term scheduler takes 0.5 ms per decision and gives each process a 4.5 ms slice. What fraction of the CPU is spent scheduling?

4. A context switch costs $s = 2$ μs. Find the overhead fraction for $q = 1$ ms and $q = 20$ μs, and the slice q at which overhead reaches exactly 10%.

5. A process calls `getpid`, then `read` on a disk file whose block is not cached, then `exit`. For each call say whether a context switch must follow, and give the state transitions.

Solutions

1. (a) Legal: the short-term scheduler selects it and the dispatcher loads its context. (b) Illegal: a woken process goes `waiting` → `ready` and must be dispatched. (c) Legal: the process itself issues an I/O request or `wait` (a system call). (d) Illegal: a process that is not running cannot issue the request it would wait on.

2. Waiting $= 120 - 8 - 17 - 2 = 93$. With 8 cores, at most 8 processes can be running at once.

3. Each cycle lasts $4.5 + 0.5 = 5$ ms, of which 0.5 ms is scheduling: $0.5/5 = 10\%$.

4. $q = 1000$ μs: $2/1002 \approx 0.20\%$. $q = 20$ μs: $2/22 \approx 9.09\%$. Setting $s/(q+s) = 0.1$ gives $q + s = 10s$, so $q = 9s = 18$ μs.

5. `getpid`: mode switch only; the process stays `running`. `read` of an uncached block: the process must wait for the disk, so `running` → `waiting` and the kernel context-switches to another process; on completion `waiting` → `ready`, and later `ready` → `running` when dispatched. `exit`: `running` → `terminated`; the process never runs again, so the kernel must switch to another process.

Concepts That Click

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 8th ed., John Wiley and Sons — Ch. 3 §3.1 (process states, p. 103) and §3.2 (process scheduling, pp. 105–110). *(Page-cited; index built 2026-10-14.)*

[2] W. Stallings, *Operating Systems: Internals and Design Principles*, 6th ed., Prentice Hall — Ch. 3 §3.2 (process states, p. 111) and §3.4 (process control, p. 135). *(Page-cited; index built 2026-10-14.)*

[3] D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 3rd ed., O'Reilly Media, 2006 — Ch. 3 (processes and the process switch, pp. 79–130). *(Page-cited; index built 2026-10-14.)*

Concepts That Click